

Week 2 - Regression with multiple var.

Multiple features (variables)

Size in feet ²	Number of bedrooms	Number of floors	Age of home in years	Price (\$) in \$1000's
x_1	x_2	x_3	x_4	
2104	5	1	45	460
$i=2 \rightarrow$ 1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

$$x_j = j^{\text{th}}$$

$$n = 4$$

$$j = 1 \dots 4$$

n = num of features

$\vec{x}_i$ = features of i^{th} training ex

$x_j^{(i)}$ = value of feature j in i^{th} training ex

Row Vector : $x^{(2)} = [1416, 3, 2, 40]$

$$x_3^{(2)} = 2$$

model

single feature $\rightarrow f_{x,b}(x) = wX + b$

multiple feature:

$$f_{w,b}(x) = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b$$

ex:

$$f_{w,b}(x) = 0.1(\text{size}) + 4(\text{bedrooms}) + \dots + 80$$

the Price will increase by 0.1 for each size

for each bedroom increase by 4

base Price "start"

model with n -features

$$f_{w,b}(x) = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

$$\vec{w} = [w_1, w_2, w_3 \dots w_n]$$

b : is num

$$\vec{x} = [x_1, x_2, x_3 \dots x_n]$$

$\vec{}$
vectors

$$f_{w,b} = \vec{w} \cdot \vec{x} + b$$

dot Product

Vectorization

writing a code (mathematical OP) using matrix and vector OP instead of using loops for better performance and efficiency

Parameters and features

$$\vec{w} = [w_1 \ w_2 \ w_3] \quad n=3$$

b is a number

$$\vec{x} = [x_1 \ x_2 \ x_3]$$

linear algebra: count from 1

$$w[0] \quad w[1] \quad w[2]$$

NumPy

```
w = np.array([1.0, 2.5, -3.3])
```

```
b = 4
```

```
x = np.array([10, 20, 30])
```

code: count from 0

Without vectorization $n=100,000$

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

```
f = w[0] * x[0] +  
     w[1] * x[1] +  
     w[2] * x[2] + b
```



Without vectorization

$$f_{\vec{w},b}(\vec{x}) = \left(\sum_{j=1}^n w_j x_j \right) + b \quad \sum_{j=1}^n \rightarrow j=1 \dots n, 1, 2, 3$$

$\text{range}(0, n) \rightarrow j=0 \dots n-1$

```
f = 0  
for j in range(0, n):  
    f = f + w[j] * x[j]  
f = f + b
```



Vectorization

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

```
f = np.dot(w, x) + b
```



Without vectorization

```
for j in range(0, 16):  
    f = f + w[j] * x[j]
```

t_0
 $f + w[0] * x[0]$

t_1
 $f + w[1] * x[1]$

...

t_{15}
 $f + w[15] * x[15]$

Vectorization

```
np.dot(w, x)
```

t_0

w[0]	w[1]	...	w[15]
------	------	-----	-------

in parallel / * * * *

x[0]	x[1]	...	x[15]
------	------	-----	-------

t_1

w[0]*x[0]	+	w[1]*x[1]	+	...	+	w[15]*x[15]
-----------	---	-----------	---	-----	---	-------------

efficient $\rightarrow$ scale to large datasets

Gradient descent $\vec{w} = (w_1 \ w_2 \ \dots \ w_{16})$ ~~b~~ parameters
derivatives $\vec{d} = (d_1 \ d_2 \ \dots \ d_{16})$

```
w = np.array([0.5, 1.3, ... 3.4])
```

```
d = np.array([0.3, 0.2, ... 0.4])
```

compute $w_j = w_j - \underbrace{0.1}_{\text{learning rate } \alpha} d_j$ for $j = 1 \dots 16$

Without vectorization

$$w_1 = w_1 - 0.1d_1$$

$$w_2 = w_2 - 0.1d_2$$

$$\vdots$$

$$w_{16} = w_{16} - 0.1d_{16}$$

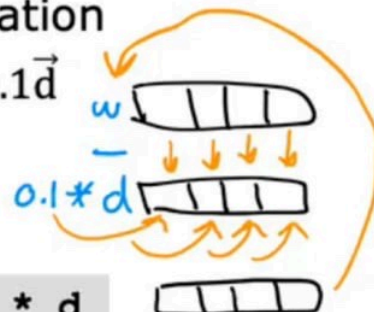
```
for j in range(0,16):
```

```
    w[j] = w[j] - 0.1 * d[j]
```

Update without
vectorization

With vectorization

$$\vec{w} = \vec{w} - 0.1\vec{d}$$



```
w = w - 0.1 * d
```

Update with
vectorization

Gradient descent for multiple LR

Parameters $\begin{matrix} \swarrow \\ w_1, w_2 \dots, w_n \\ \searrow \\ b \end{matrix}$

$$\text{model } f_{\vec{w}, b}(\vec{x}) = w_1 x_1 + \dots + w_n x_n + b$$

Cost fun $J(w_1, \dots, w_n, b)$

Gradient descent:

$$w_j = w_j - \epsilon \times \frac{\partial}{\partial w_j} J(w_1, \dots, w_n, b)$$

$$b_j = b_j - \epsilon \frac{\partial}{\partial b_j} J(w_1, \dots, w_n, b)$$

Vector notation

$$\vec{w} = [w_1, \dots, w_n]$$



b : is number

$$f_{\vec{w}, b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

Cost function $J(\vec{w}, b)$

Gradient descent

$$w_j = w_j - \epsilon \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b_j = b_j - \epsilon \frac{\partial}{\partial b_j} J(\vec{w}, b)$$

GD - one feature

$$w_j = w_j - \eta \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x_i) - y_i) x_i$$

$$b_j = b_j - \eta \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x_i) - y_i)$$

GD - n-features ($n \geq 2$)

$j=1$

$$w_1 = w_1 - \eta \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}_i) - y_i) x_1^{(i)}$$

↓

$$\frac{\partial}{\partial w_1} J(\vec{w}, b)$$

$j=n$

$$w_n = w_n - \eta \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}_i) - y_i) x_n^{(i)}$$

$$b = b - \eta \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}_i) - y_i)$$

An alternative to gradient descent

Normal equation

- only for LR
- solve w, b without GD

Cons:

- only works for LR
- slow if features is large $> 10,000$

l2b C1_w2

$\vec{x}_{\text{train}}$
matrix

$$= \begin{bmatrix} 2104 & 5 & 1 & 45 \\ 1416 & 3 & 2 & 40 \\ 852 & 2 & 1 & 35 \end{bmatrix}$$


containing examples $\rightarrow$ each row represents
one example

$$m = 3$$

$$n = 4$$

$$(3, 4)$$

rows columns

y_{train}
vector

$$= [460, 232, 178] \rightarrow (3,)$$

Parameter w, b $n=4$

w is a vector with n elements

- Each element contains parameter associated with one feature

$$\vec{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{n-1} \end{bmatrix}$$

b is a scalar (number)

1-Model Prediction with n -features

model: $f_{w,b}(x) = \vec{w} \cdot x + b$

w -init = vector with n elements

b -init = number

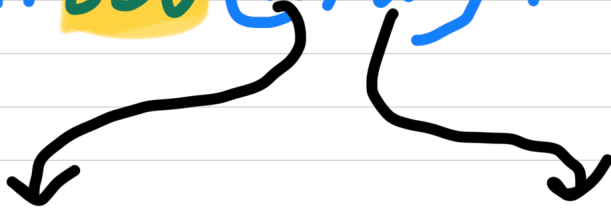
x -vec = x -train $[0, :]$ $\rightarrow [2104, 5, 1, 45]$



using vectorization

def pred(x, w, b)

np.dot(x, w) + b



$$([2104, 5, 1, 45] \bullet [1, 2, 2, 4]) + b$$

$$= 459, 1 \rightarrow \text{from lab}$$

2- Compute cost with mul var

cost: $J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x_i) - \hat{y}_i)^2$

$$f_{w,b}(x_i) = w \cdot x + b$$

def compute_cost(x, y, w, b)

m = x.shape[0]

cost = 0

for i in range(m):

f_wb_i = np.dot(x[i], w) + b

cost = cost + (f_wb_i - y[i])²

cost = $\frac{\text{cost}}{2 \times m}$

Return cost

Gradient descent with mul. var.

$$w_j = w_j - \eta \frac{\partial J(w, b)}{\partial w_j} \rightarrow \text{cost fun}$$

$$b = b - \eta \frac{\partial J(w, b)}{\partial b}$$

n : num of features

m : is num of training

Examples

$$\frac{\partial J(w, b)}{\partial w_j} = \frac{1}{m} \sum_{i=0}^m (f_{w, b}(x_i) - y_i) x_i$$

$$\frac{\partial J(w, b)}{\partial b} = \frac{1}{m} \sum_{i=0}^m (\overbrace{f_{w, b}(x_i) - y_i}^{y_i})$$

```
def compute_gradient(x, y, w, b, compute_cost):  
  
    m, n = x.shape[0] # m examples , n features  
  
    dj_dw = np.zeros((n,)) # create 1d array [0,0,0,0]  
    dj_db = 0  
  
    for i in range(m): # (m) examples  
        error = (np.dot(x[i], w) + b) - y[i] # computes the error  $y^i - y_i$   
  
        for j in range(n): # (n) features  
            dj_dw[j] += error * x[i, j] # partial derivative  $dj_{dw}$   
  
        dj_db += error # partial derivative  $dj_{db}$   
  
    dj_dw = dj_dw / m  
    dj_db = dj_db / m  
  
    return dj_dw, dj_db
```

Implementing:

$$w_j = w_j - \alpha \frac{\partial J(w, b)}{\partial w_j} \rightarrow \text{cost fun}$$

$$b = b - \alpha \frac{\partial J(w, b)}{\partial b}$$

```
def gradient_descent(x, y, w_in, b_in, compute_cost, compute_gradient, alpha, num_iters):  
  
    j_history = []  
    w = copy.deepcopy(w_in)  
    b = b_in  
  
    for i in range(num_iters):  
        dj_dw, dj_db = compute_gradient(x, y, w, b)  
        # parameter updates  
        w = w - alpha * dj_dw  
        b = b - alpha * dj_db  
  
        if i < 100000 : # cost history  
            j_history.append(compute_cost(x, y, w, b))  
  
        if i % math.ceil(num_iters/10) == 0:  
            print(f"Iteration {i:4d}: Cost {j_history[-1]:8.2f}")  
  
    return w, b
```

Feature Scaling

- Preprocessing step
- Transform the input features to be on similar

ex : x_1 : size x_2 : num bedroom
Rent: 300-2000 0-5

$$\text{Price} = w_1 x_1 + w_2 x_2 + b$$

$\downarrow$ $\downarrow$
size bedrooms

House : $x_1 = 200$, $x_2 = 5$, Price = 500k

Size of parameters w_1, w_2

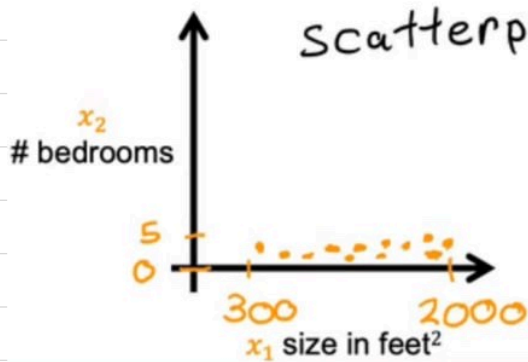
$w_1 = 50$, $w_2 = 0.1$, $b = 50$	$w_1 = 0.1$, $w_2 = 50$, $b = 50$
Price = 100,050,5 k	Price = 500k
	more reasonable

Feature size and parameter size

	size of feature x_j	size of parameter w_j
size in feet ²		
#bedrooms		

Features

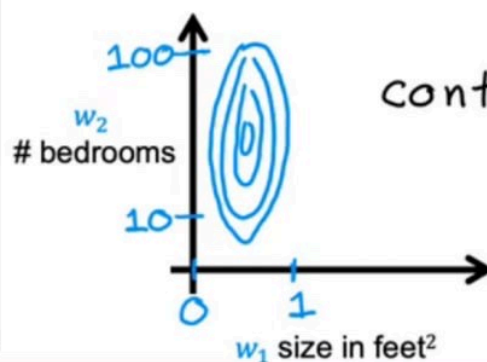
scatterplot



Parameters

$J(\vec{w}, b)$

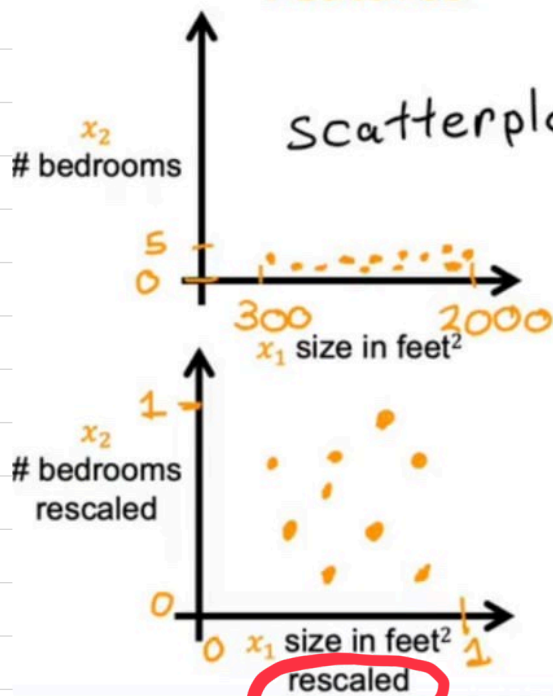
contour plot



Feature size and gradient descent

Features

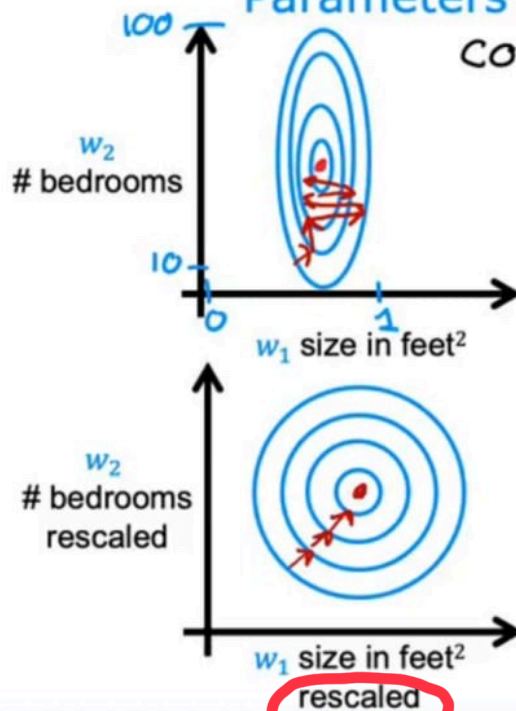
scatterplot



Parameters

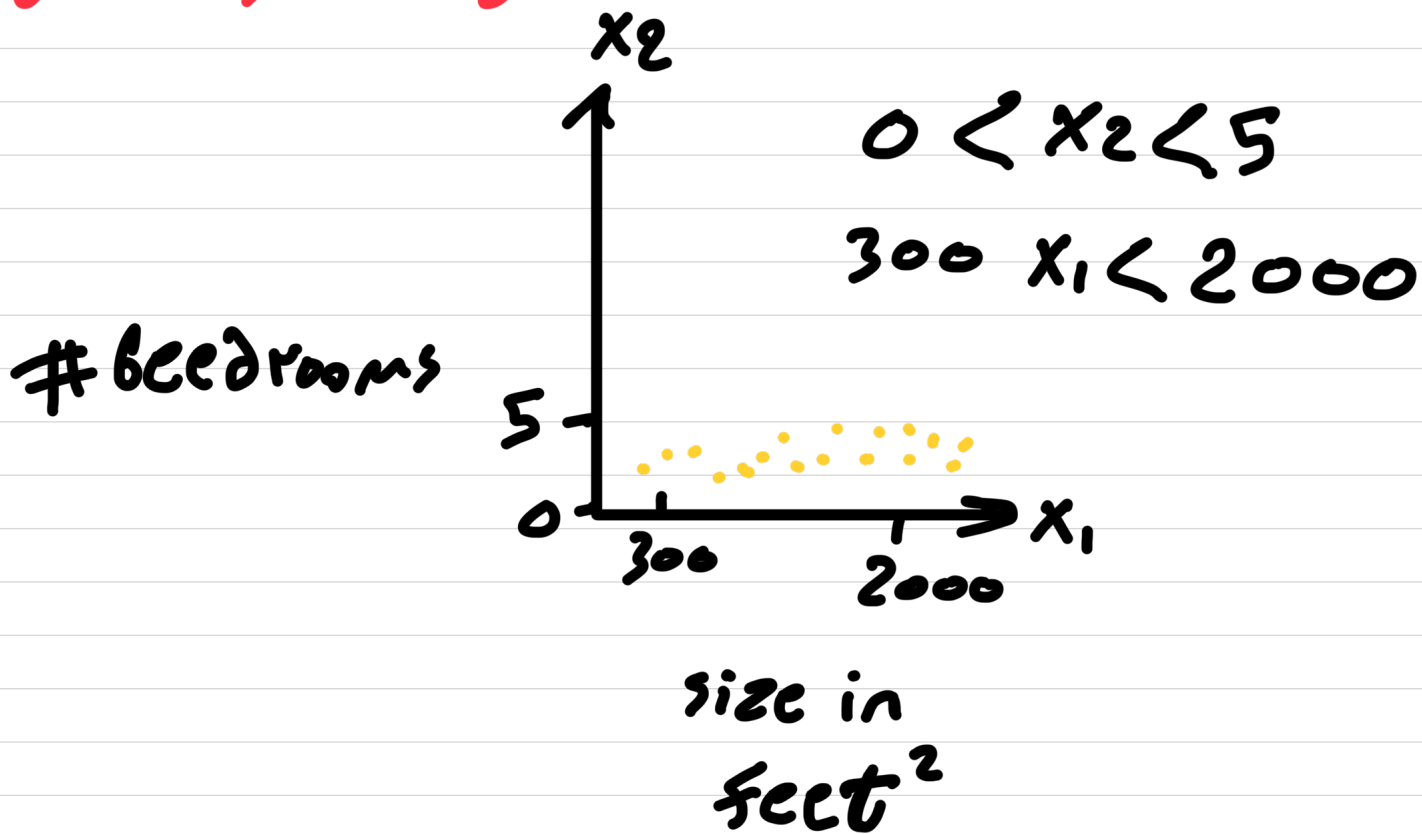
contour plot

$J(\vec{w}, b)$



GD can find
direct path
to global
min

feature scaling



1-max Normalization

$$x_{1\text{-scaled}} = \frac{x_1}{2000}$$

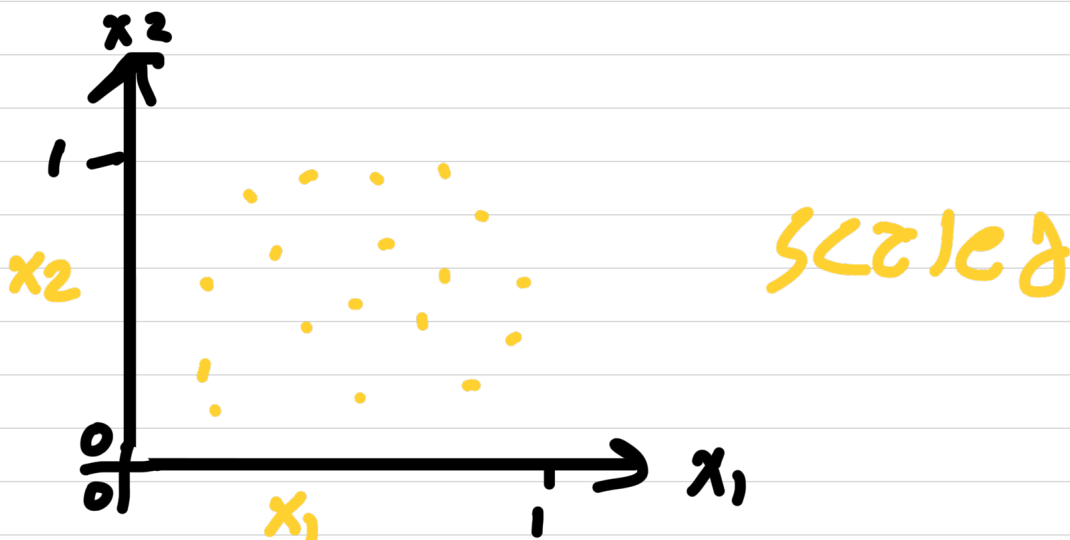
↓
max

$$0.15 < x_{1\text{-scaled}} < 1$$

$$x_{2\text{-scaled}} = \frac{x_2}{5}$$

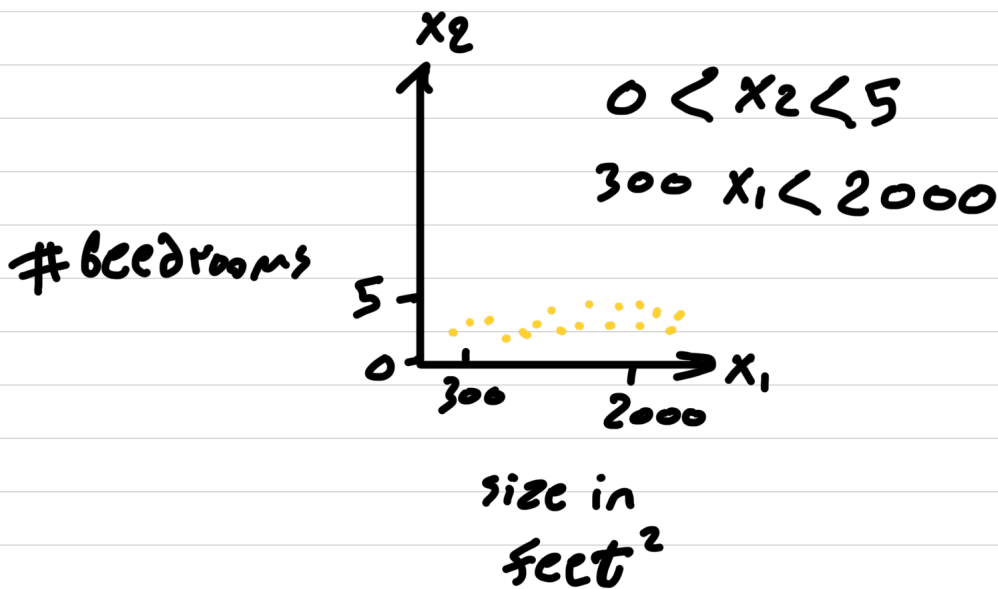
↓
max

$$0 \leq x_{2\text{-scaled}} \leq 1$$



2- **mean normalization** : $x_{scaled} = \frac{x - \mu}{max - min}$

- μ : Avg of feature
- max, min : max, min value of features



1- find the **Avg** of x_1 , x_2
 μ $\downarrow$ $\downarrow$
 μ_1 μ_2

$$x_1 = \frac{x_1 - \mu_1}{max - min} = \frac{x_1 - 600}{2000 - 300}$$

$$-0.18 \leq x_1 \leq 0.82$$

for x_2 $\frac{x_2 - 2.3}{5 - 0} = -0.46 \leq x_2 \leq 0.54$

3- Z-score : $x_{scaled} = \frac{x - \mu}{\sigma}$

- μ : mean of the feature

- σ : STD of feature



• for feature scaling Aim for about

• $-1 \leq x_j \leq 1$

• $-3 \leq x_j \leq 3$

• $-0,3 \leq x_j \leq 0,3$

acceptable
Ranges

• $0 \leq x_j \leq 3$

• $-2 \leq x_j \leq 0,5$

okay, no rescaling

• $-100 \leq x_3 \leq 100$ too large $\rightarrow$ scale

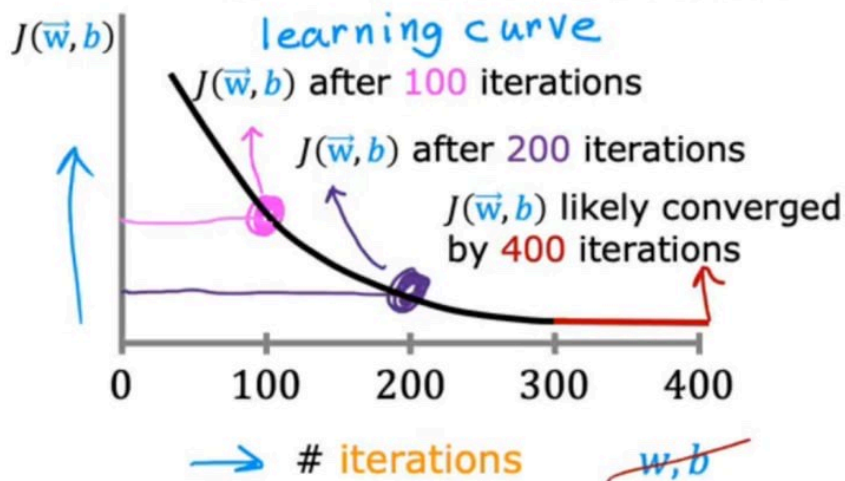
• $-0,001 \leq x_4 \leq 0,001$ too small $\rightarrow$ scale

It will help the GD to run faster

make sure the GD working correctly

Make sure gradient descent is working correctly

objective: $\min_{\bar{w}, b} J(\bar{w}, b)$ $J(\bar{w}, b)$ should **decrease** after every iteration



iterations needed varies 30 1,000 100,000

Automatic convergence test

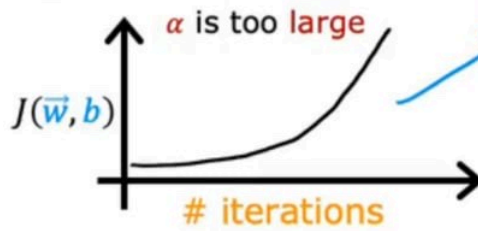
Let ϵ "epsilon" be 10^{-3} .
0.001

If $J(\bar{w}, b)$ decreases by $\leq \epsilon$ in one iteration, declare **convergence**.

(found parameters $\bar{w}, b$ to get close to global minimum)

choosing the learning rate " α "

Identify problem with gradient descent



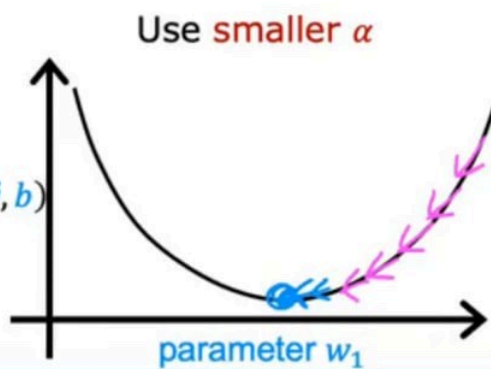
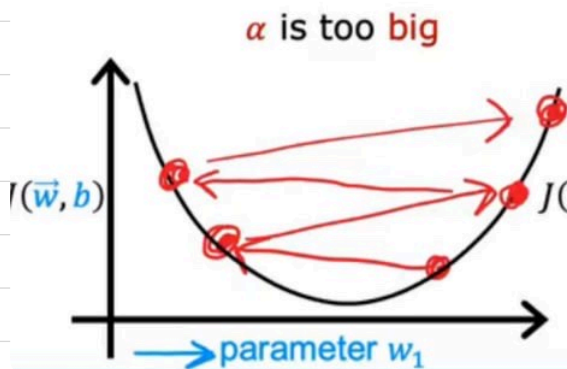
or learning rate is too large

$$w_1 = w_1 + \alpha d_1$$

use a minus sign

$$w_1 = w_1 - \alpha d_1$$

Adjust learning rate

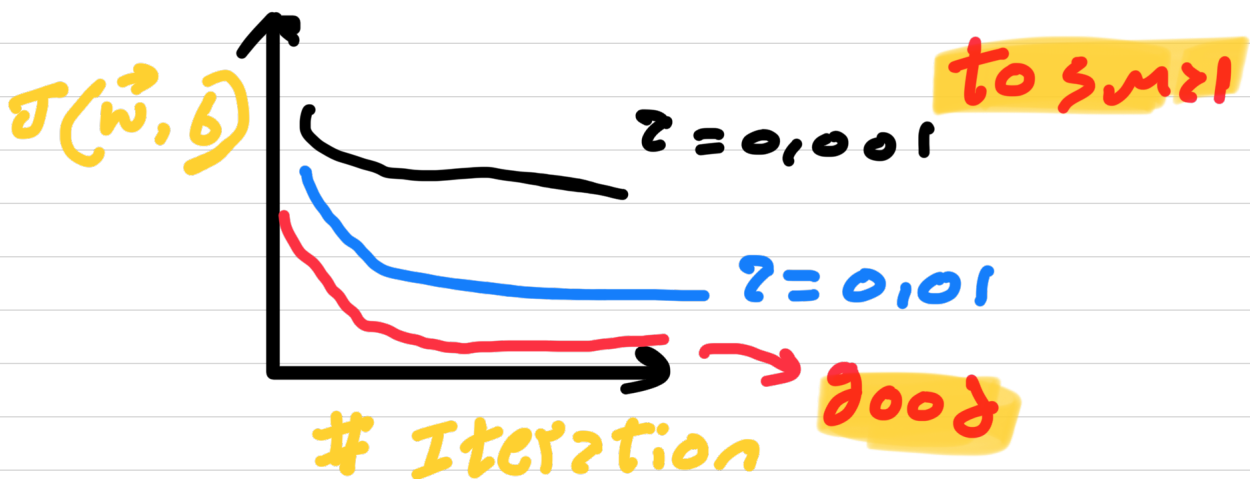


With a small enough α , $J(\vec{w}, b)$ should decrease on every iteration

If α is too small, gradient descent takes a lot more iterations to converge

Values of α to try:

• 0,001 • 0,01 • 0,1 • 1



feature engineering



feature engineering : Using intuition to design new features, by transforming or combining original features

$$\text{ex: } f_{\vec{w},b}(\vec{x}) = w_1 x_1 + w_2 x_2 + b$$

$\downarrow$ width (frontage) $\downarrow$ Depth

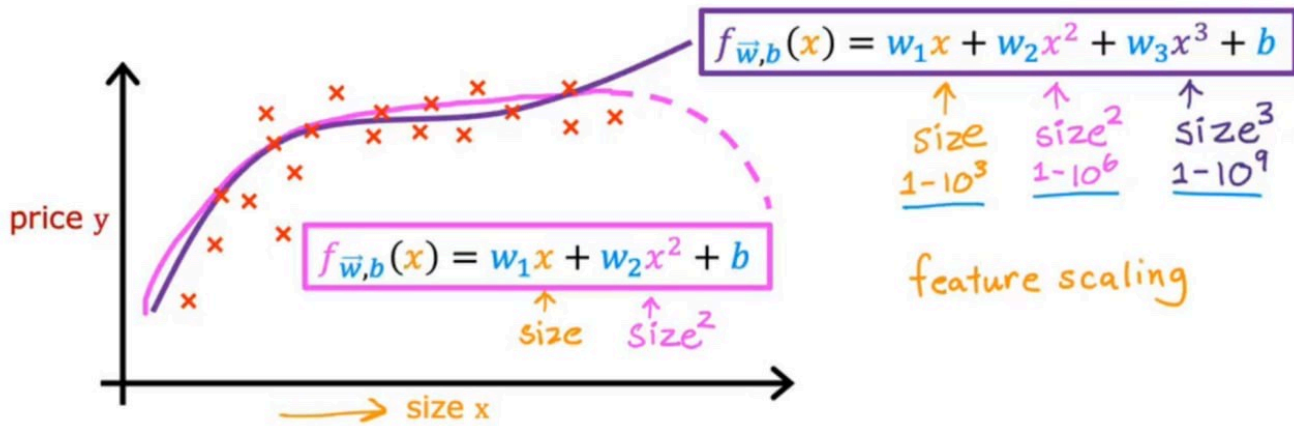
$$\text{Area} = \text{width} * \text{Depth}$$

$$x_3 = x_1 * x_2 \quad \text{new feature}$$

$$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

Polynomial Regression

Polynomial regression



Choice of features

